

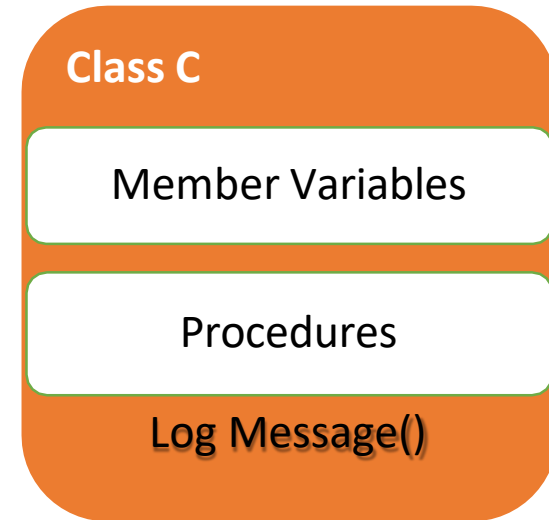
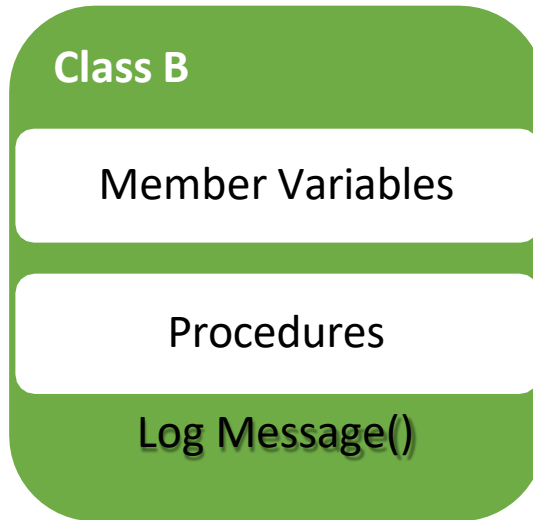
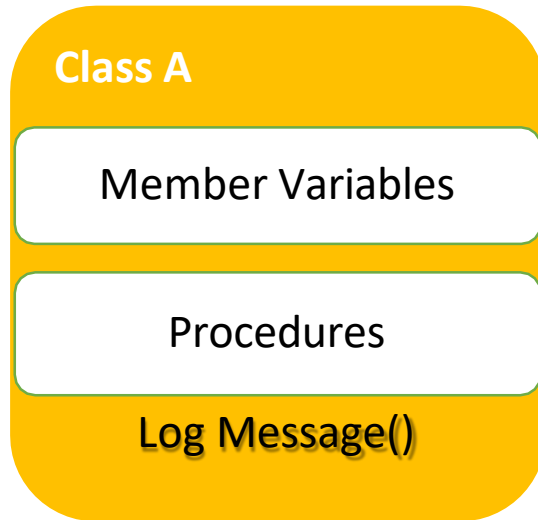


Advance Software Engineering Aspect-Oriented Programming

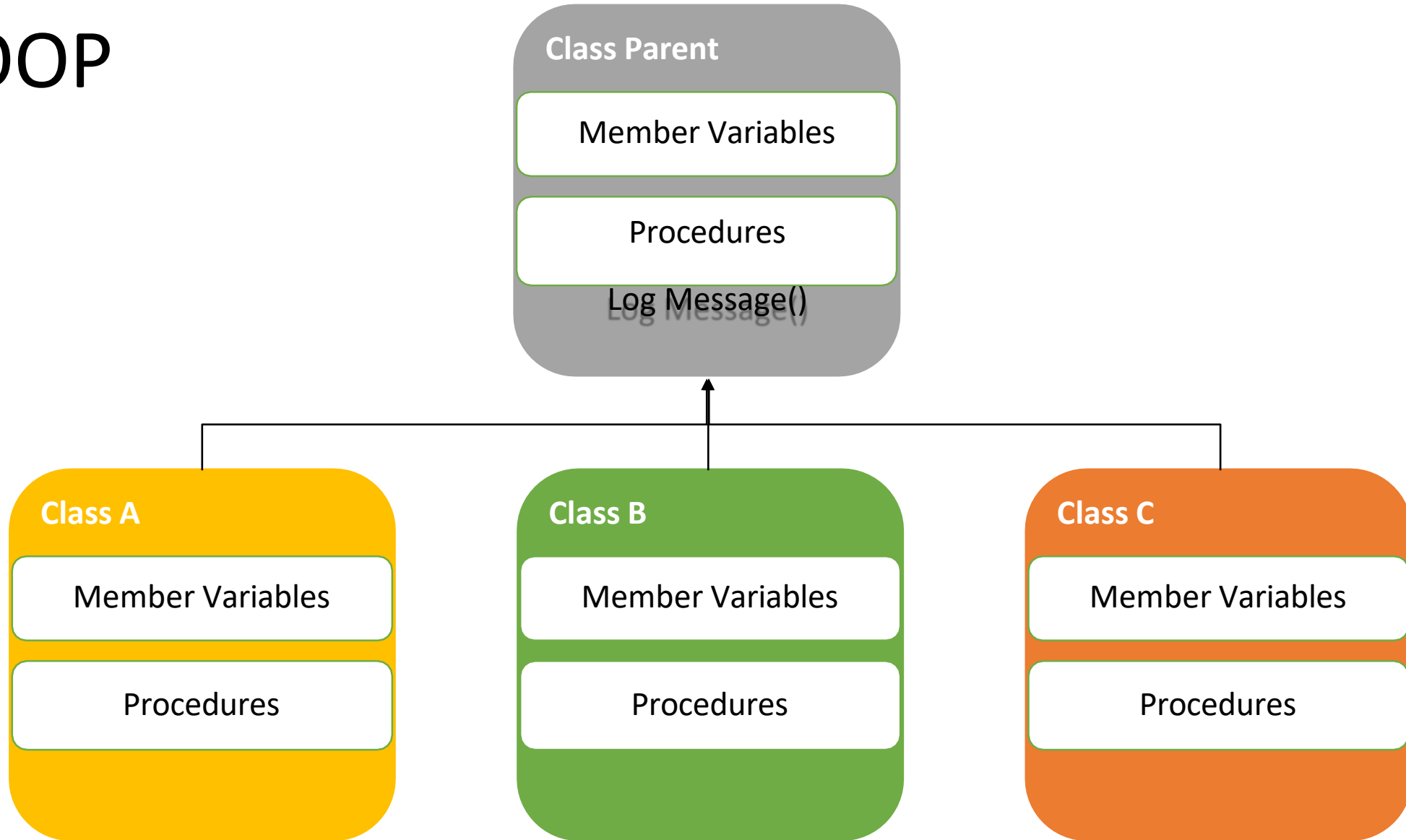
By:
Dr. Salwa Osama

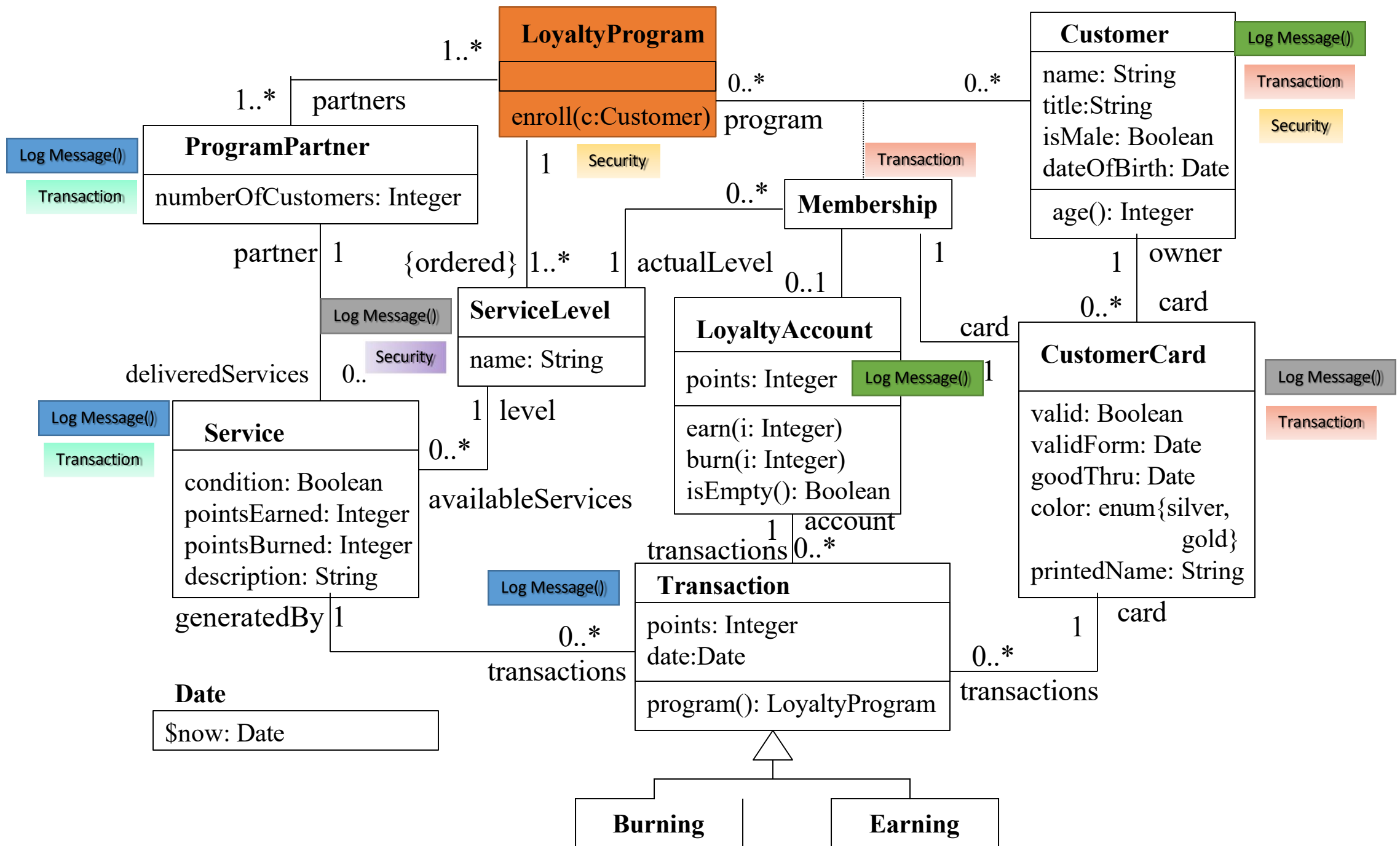


OOP



OOP





Privilege

Member Variables

Procedures

Logger

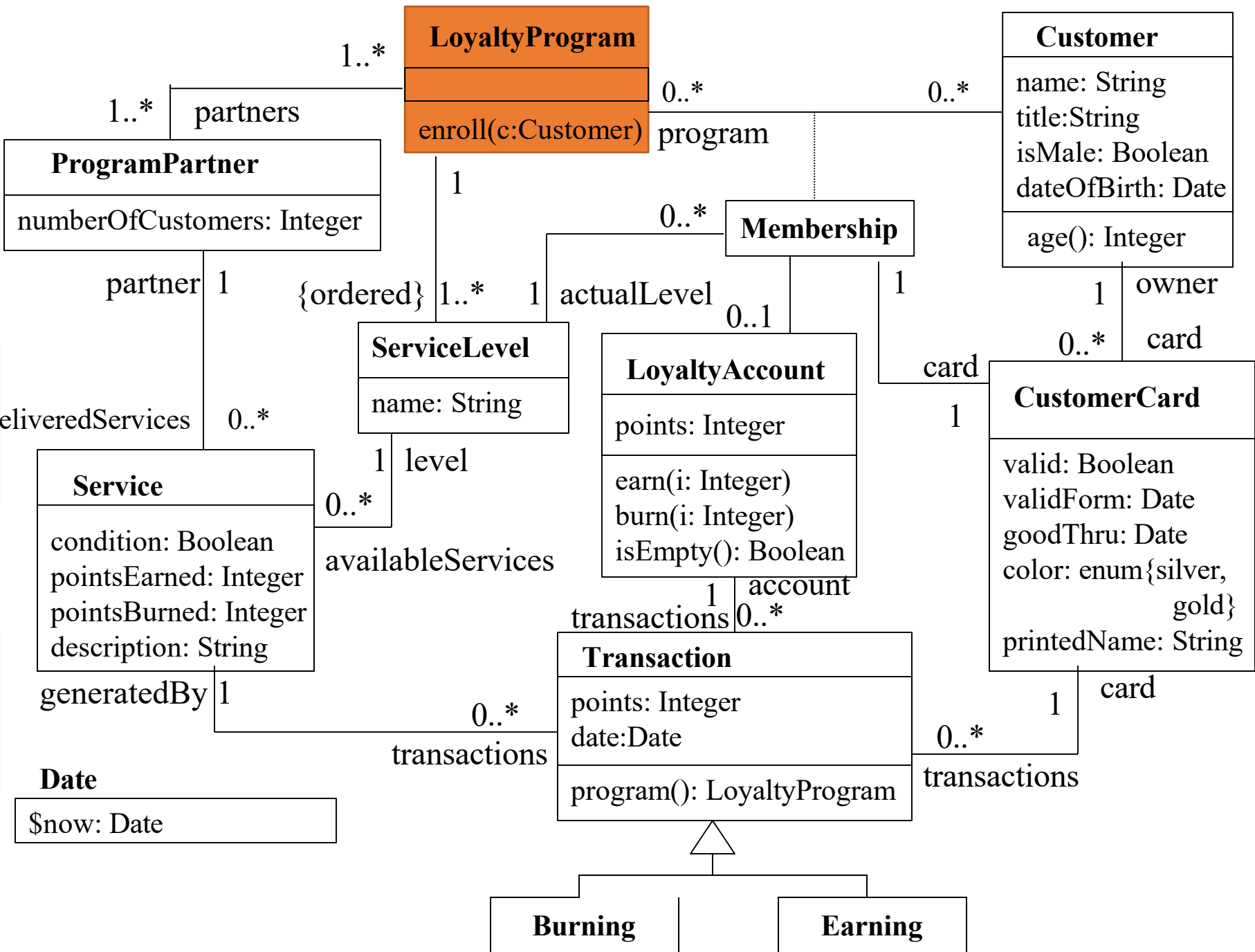
Member Variables

Procedures

DB

Member Variables

Procedures





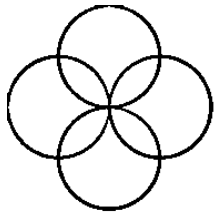
Cuts across multiple abstractions



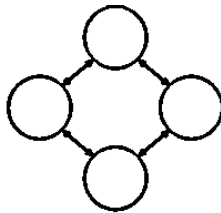
Boilerplate code



Difficult to decompose



Tight Coupling



Loose Coupling

High-coupling



PROBLEM

Problems



Code tangling

Code scattering

- **Poor traceability** – multiple concerns in the same module breaks linkage between requirement and its implementation, making it harder to understand what a piece of code is doing to address the problem domain.
- **Lower productivity** – developers are spending too much time and attention to peripheral issues rather than the problem domain.
- **Less code reuse** – boilerplate code propagated through cut and paste... no code reuse here!
- **Harder refactoring** – changing requirements means touching many modules for a single concern.

Aspect Oriented Programming



Aspect Oriented Programming (AOP)

Before AOP



After AOP



Concerns

- Concerns are not program issues but reflect the system requirements and the priorities of the system stakeholders.
 - Examples of concerns are **performance, security, specific functionality**, etc.
- **By reflecting the separation of concerns** in a program, there is clear traceability from requirements to implementation.
- **Core concerns** are the functional concerns that relate to the primary purpose of a system; **secondary concerns** are functional concerns that reflect non-functional requirements.

What is "Separation of Concerns"?

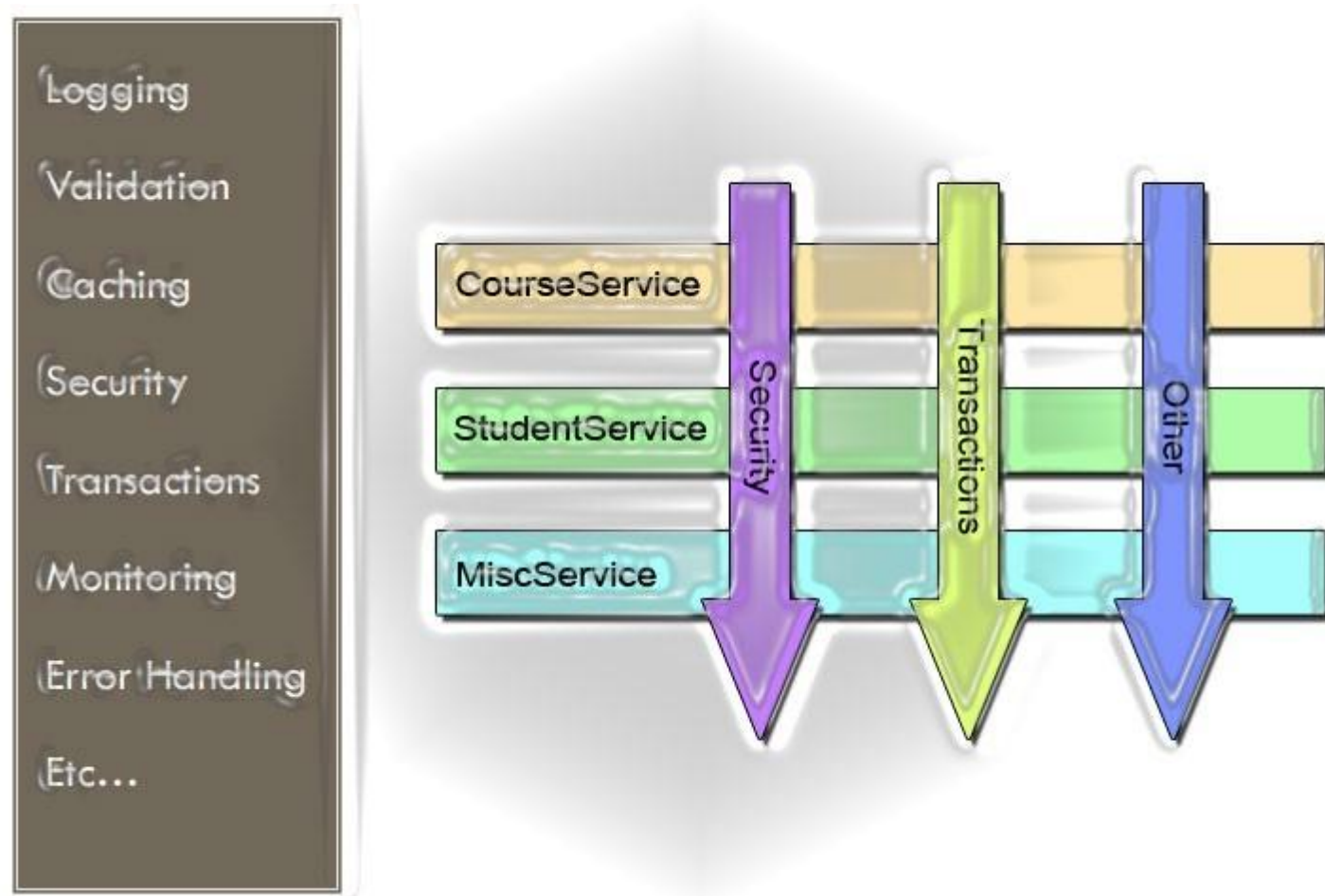
It means splitting your program into separate parts, where each part handles one specific concern (like security, logging, performance, etc.) — instead of mixing everything together.

Stakeholder concerns

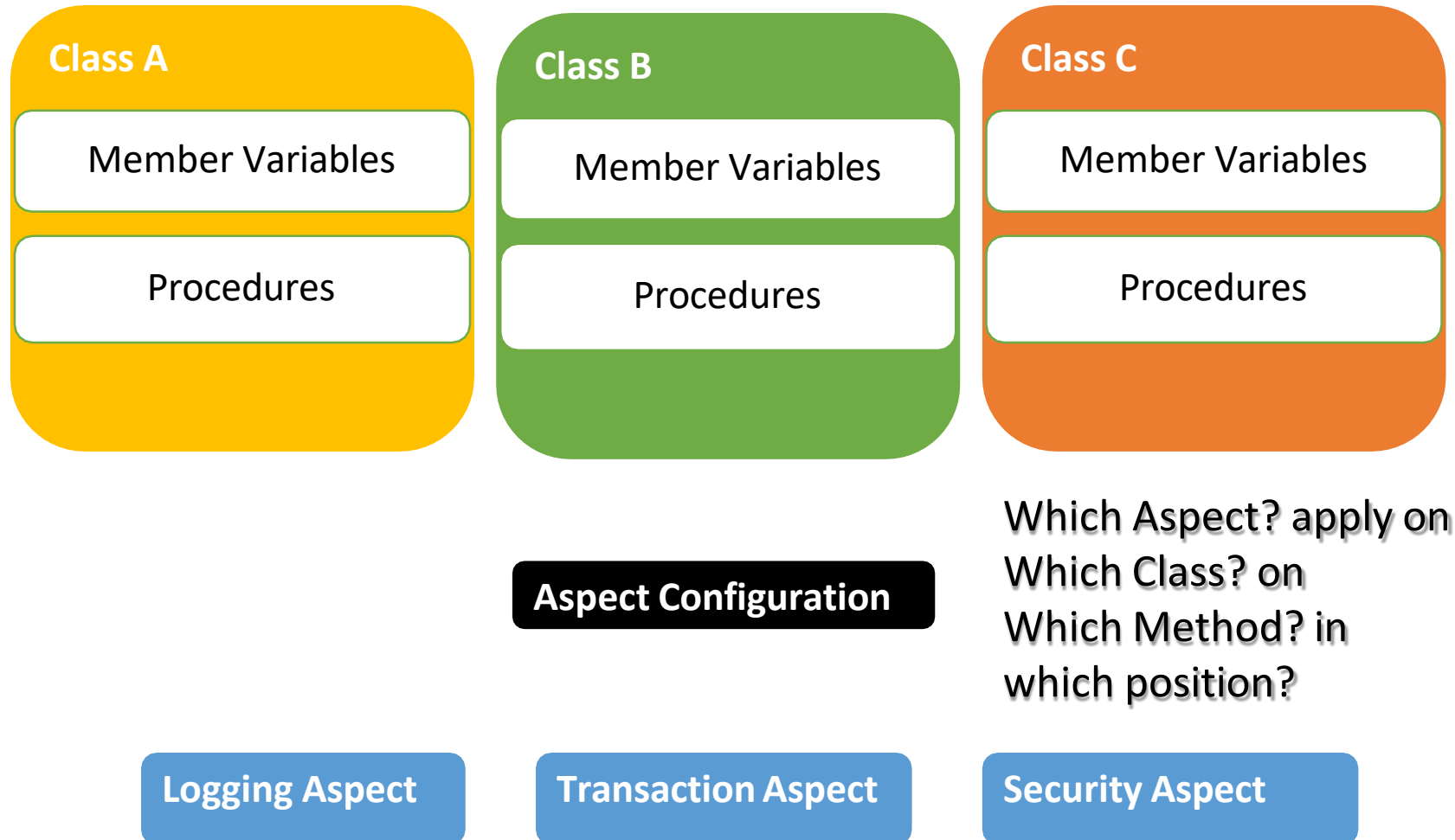


- **Functional** concerns which are related to specific functionality to be included in a system.
- **Quality** of service concerns which are related to the non-functional behaviour of a system.
- **Policy** concerns which are related to the overall policies that govern the use of the system.
- **System** concerns which are related to attributes of the system as a whole such as its maintainability or its configurability.
- **Organisational** concerns which are related to organisational goals and priorities such as producing a system within budget, making use of existing software assets or maintaining the reputation of an organisation.

Cross-cutting Concerns



AOP



AOP Demo

```
public Book GetBookById(Guid id) {  
    if (id == default(Guid)) {  
        throw new ArgumentException("Invalid book id", "id");  
    }  
  
    Logger.DebugFormat("Getting book id {0}", id);  
  
    try {  
        var book = Dal.Get<Book>(id);  
  
        Logger.DebugFormat("GetBookById returned with book [{0}]", book);  
  
        return book;  
    } catch (Exception ex) {  
        Logger.Error("Oops, could not get book by id!", ex);  
  
        throw;  
    }  
}
```



AOP Demo



```
/// <summary>
/// Simple repository using AOP attributes
/// </summary>
// all public methods starting with 'Get'
[CheckParameters(AttributeTargetElements = MulticastTargets.Method,
                  AttributeTargetMemberAttributes = MulticastAttributes.Public,
                  AttributeTargetMembers = "Get*",
                  AspectPriority = 2)]

// apply to all methods
[Trace(AttributeTargetElements = MulticastTargets.Method,
        AspectPriority = 1)]
public class AopRepository : IRepository {
    public AopRepository(IDataAccessLayer dal) {
        Dal = dal;
    }

    protected IDataAccessLayer Dal { get; set; }

    public Book GetBookById([NotDefaultGuid] Guid id) {
        return Dal.Get<Book>(id);
    }

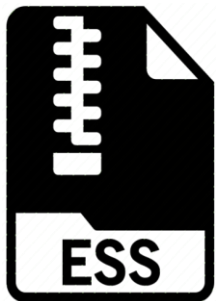
    public Author GetAuthorById([NotDefaultGuid] Guid id) {
        return Dal.Get<Author>(id);
    }
}
```

Benefits

- Centralize concerns implementation
- Intercept method calls
- Inject new behavior
- More reusable code
- Cleaner code



What's in it for **YOU**?



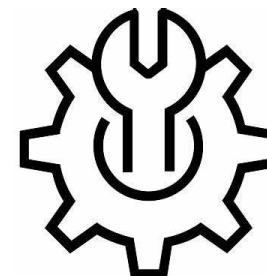
Write less code



Read less code



More concise and easy to understand



More maintainable



AOP Terminologies

```
4 import org.apache.commons.lang3.StringUtils;
5 import org.aspectj.lang.JoinPoint;
6 import org.aspectj.lang.ProceedingJoinPoint;
7 import org.aspectj.lang.annotation.Around;
8 import org.aspectj.lang.annotation.Aspect;
9 import org.slf4j.Logger;
10 import org.slf4j.LoggerFactory;
11 import org.springframework.stereotype.Component;
12
13 @Aspect
14 @Component
15 public class LogTimeAspect {
16
17
18     Logger log = LoggerFactory.getLogger(LogTimeAspect.class);
19
20     @Around(value = "execution(* com.global.book.service..*(..))")
21     public Object logTime(ProceedingJoinPoint joinPoint) throws Throwable {
22
23         long startTime = System.currentTimeMillis();
```

Aspect

- It is a class that implements enterprise application concerns that cut across multiple classes such as transaction management.
- Aspects can be a normal class configuration through the spring XML configuration or we can use spring AspectJ integration to define class as Aspect using @aspect annotation

AOP Terminologies

```
13 @Aspect
14 @Component
15 public class LogTimeAspect {
16
17     Logger log = LoggerFactory.getLogger(LogTimeAspect.class);
18
19     @Around(value = "execution(* com.global.book.service..*(..))")
20     public Object logTime(ProceedingJoinPoint joinPoint) throws Throwable {
21
22         long startTime = System.currentTimeMillis();
23         StringBuilder sb = new StringBuilder("KPI:");
24         sb.append("[").append(joinPoint.getKind()).append("]\tfor: ").append(joinPoint.getSignature())
25             .append("\twithArgs: ").append("(").append(StringUtils.join(joinPoint.getArgs(), ","))
26             .append(")").append("\ttook: ");
27         Object returnValue = joinPoint.proceed();
28         log.info(sb.append(System.currentTimeMillis() - startTime).append(" ms.").toString());
29
30         return returnValue;
31     }
32 }
33
34 }
```

Problems Javadoc Declaration Search Console Terminal Servers

data-jpa-books-project - DataJpaBooksProjectApplication [Spring Boot App] D:\Global\Tools\sts-4.12.1.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_17.0.0.v20211012-1059\jre\bin\java

```
] for: void com.global.book.service.PriceSchedule.computePrice() withArgs: () took: 4008 ms.
] for: void com.global.book.service.PriceSchedule.computeDiscount() withArgs: () took: 4007 ms.
-02-01T19:04:49.033621700
022-02-01T19:04:49.033621700
] for: void com.global.book.service.PriceSchedule.computeDiscount() withArgs: () took: 4014 ms.
] for: void com.global.book.service.PriceSchedule.computePrice() withArgs: () took: 4013 ms.
022-02-01T19:08:09.047246100
-02-01T19:08:09.047246100
] for: void com.global.book.service.PriceSchedule.computeDiscount() withArgs: () took: 4011 ms.
] for: void com.global.book.service.PriceSchedule.computePrice() withArgs: () took: 4011 ms.
```

- Join point:
 - It is a specific point in the application such as method execution, exception handling, changing object variable values. In Spring AOP, a join point is always the execution of the method.

AOP Terminologies

```
13 @Aspect
14 @Component
15 public class LogTimeAspect {
16
17     Logger log = LoggerFactory.getLogger(LogTimeAspect.class);
18
19     @Around(value = "execution(* com.global.book.service..*(..))")
20     public Object logTime(ProceedingJoinPoint joinPoint) throws Throwable {
21
22         long startTime = System.currentTimeMillis();
23         StringBuilder sb = new StringBuilder("KPI:");
24         sb.append("[").append(joinPoint.getKind()).append("]\tfor: ").append(joinPoint.getSignature())
25             .append("\twithArgs: ").append("(").append(StringUtils.join(joinPoint.getArgs(), ",")).append(")");
26         sb.append("\tttook: ");
27         Object returnValue = joinPoint.proceed();
28         log.info(sb.append(System.currentTimeMillis() - startTime).append(" ms.").toString());
29
30         return returnValue;
31     }
32 }
33
34 }
```

Problems Javadoc Declaration Search Console Terminal Servers

ata-jpa-books-project - Data\paBooksProjectApplication [Spring Boot App] D:\Global\Tools\sts-4.12.1.RELEASE\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_17.0.0.v20211012-1059

for: List com.global.book.service.PostService.getAllPost() withArgs: () took: 311 ms.

ssor: Using 'application/json', given [*/] and supported [application/json, application/*+json, application/json, appl

, title=sunt aut facere repellat provident occaecati excepturi optio reprehenderit, (truncated)...]

={}]

global.book.controller.PostController#getPostById(Long)

older.typicode.com/posts/1

pplication/cbor, application/*+json]

.entity.PostDto]

for: PostDto com.global.book.service.PostService.postById(Long) withArgs: (1) took: 375 ms.

- Advice
 - It are actions taken for a particular joint point

AOP Advice Types

- **Before Advice:** These advices runs before the execution of join point methods. We can use `@Before` annotation to mark an advice type as Before advice.
- **After (finally) Advice:** An advice that gets executed after the join point method finishes executing, whether normally or by throwing an exception. We can create after advice using `@After` annotation.
- **After Returning Advice:** Sometimes we want advice methods to execute only if the join point method executes normally. We can use `@AfterReturning` annotation to mark a method as after returning advice.
- **After Throwing Advice:** This advice gets executed only when join point method throws exception, we can use it to `rollback` the transaction declaratively. We use `@AfterThrowing` annotation for this type of advice.
- **Around Advice:** This is the most important and powerful advice. This advice surrounds the join point method and we can also choose whether to execute the join point method or not. We can write advice code that gets executed before and after the execution of the join point method. It is the responsibility of around advice to invoke the join point method and return values if the method is returning something. We use `@Around` annotation to create around advice methods.

AOP Advice Types

```
public void beforeGetPosts(JoinPoint joinPoint){
    System.out.println("Before fetching posts: "+joinPoint.getSignature()+"", search = "+joinPoint.getArgs()[0]);
}

@After("execution(* com.example.pdf.demo.controllers.*.getPost(..))")
public void afterGetPosts(JoinPoint joinPoint){
    System.out.println("After fetching posts: "+joinPoint.getSignature()+"", search = "+joinPoint.getArgs()[0]);
}

@AfterReturning(pointcut = "execution(* com.example.pdf.demo.controllers.*.getPost(..))", returning = "result")
public void afterGetPostsReturning(JoinPoint joinPoint, Object result){
    System.out.println("After fetching posts: "+joinPoint.getSignature()+"", result = "+result);
}

@AfterThrowing(pointcut = "execution(* com.example.pdf.demo.controllers.*.getPost(..))", throwing = "ex")
public void afterGetPostsThrowing(JoinPoint joinPoint, Exception ex){
    System.out.println("Exception thrown while fetching posts: "+joinPoint.getSignature()+"", ex = "+ex.getMessage());
}
```

```
@Around("execution(* com.example.pdf.demo.controllers.*.getPost(..))")
public Object aroundGetPost(ProceedingJoinPoint joinPoint) throws Throwable{
    System.out.println("Before fetching post by id: "+joinPoint.getArgs()[0]);
    Object result = joinPoint.proceed();
    System.out.println("After fetching post by id. result="+result);
    return result;
}
```

```
@Around("execution(* com.example.pdf.demo.controllers.*.getPost(..))")
public void aroundGetPost(ProceedingJoinPoint joinPoint) throws Throwable{
    System.out.println("Before fetching post by id: "+joinPoint.getArgs()[0]);
    Object result = joinPoint.proceed();
    System.out.println("After fetching post by id. result="+result);
    // return result;
}
```

AOP Terminologies

- `@Pointcut("execution(* com.tutorialspoint.*(..))")`
- `@Pointcut("execution(* com.tutorialspoint.Student.getName(..))")`

```
13 @Aspect
14 @Component
15 public class LogTimeAspect {
16
17
18     Logger log = LoggerFactory.getLogger(LogTimeAspect.class);
19
20     @Around(value = "execution(* com.global.book.service..*(..))")
21     public Object logTime(ProceedingJoinPoint joinPoint) throws Throwable {
22
23         long startTime = System.currentTimeMillis();
24         StringBuilder sb = new StringBuilder("KPI:");
25         sb.append("[").append(joinPoint.getKind()).append("]\tfor: ").append(joinPoint.getSignature())
26             .append("\twithArgs: ").append("(").append(StringUtils.join(joinPoint.getArgs(), ","))
27             .append(")");
28         sb.append("\tttook: ");
29         Object returnValue = joinPoint.proceed();
30         log.info(sb.append(System.currentTimeMillis() - startTime).append(" ms.").toString());
31
32         return returnValue;
33     }
34 }
35
```

- Point cut

- It is expressions that are matched with joint point to determine whether advice needed to be executed or not
- Identifies the specific events with which advice should be associated.
- AOP is the ability to multicast the same aspects into many places in your code, this feature is called *Point Cut*

Point Cut

✓ `execution(...)`

This specifies that the **pointcut** applies to method execution join poi

✓ `*` (the first one)

This means:

Match any return type (e.g., `void`, `int`, `String`, etc.)

✓ `*` (the second one)

This means:

Match any method name (e.g., `save`, `getData`, `calculateTotal`,

✓ `(..)`

This means:

Match any number of parameters of any type (including no parar

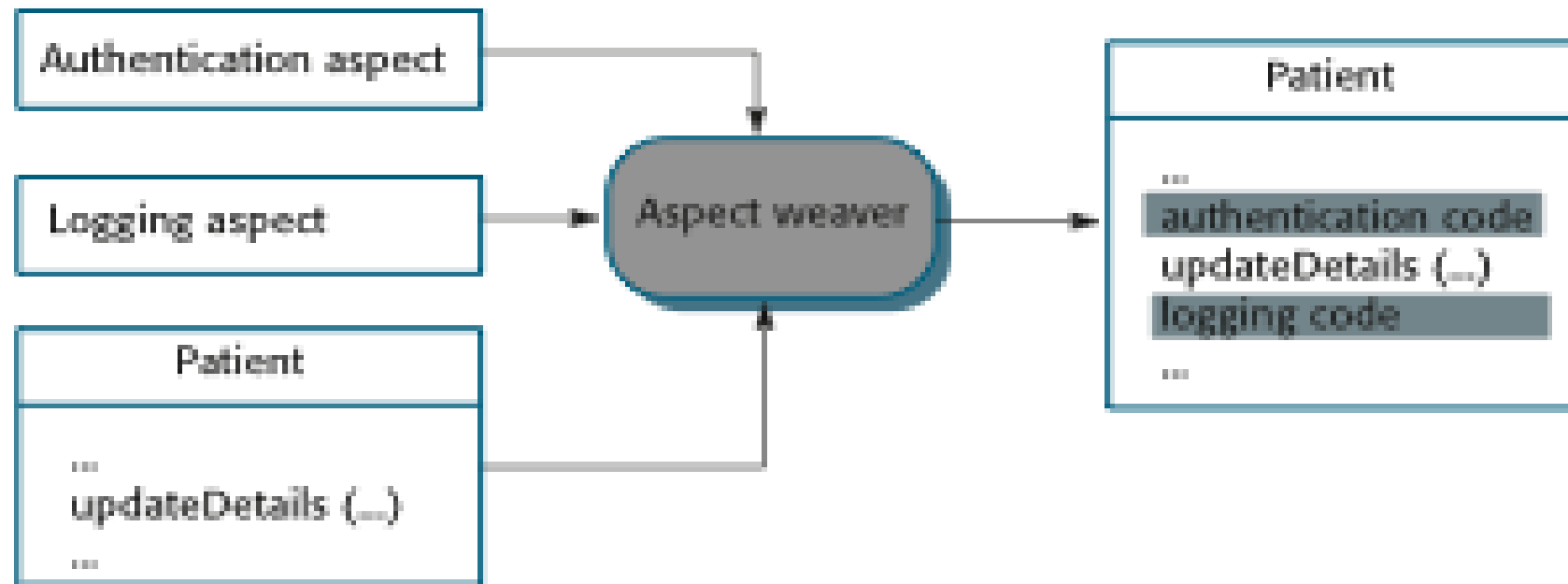
AOP Terminologies

- `@Pointcut("execution(* com.tutorialspoint.*(..))")`
- `@Pointcut("execution(* com.tutorialspoint.Student.getName(..))")`

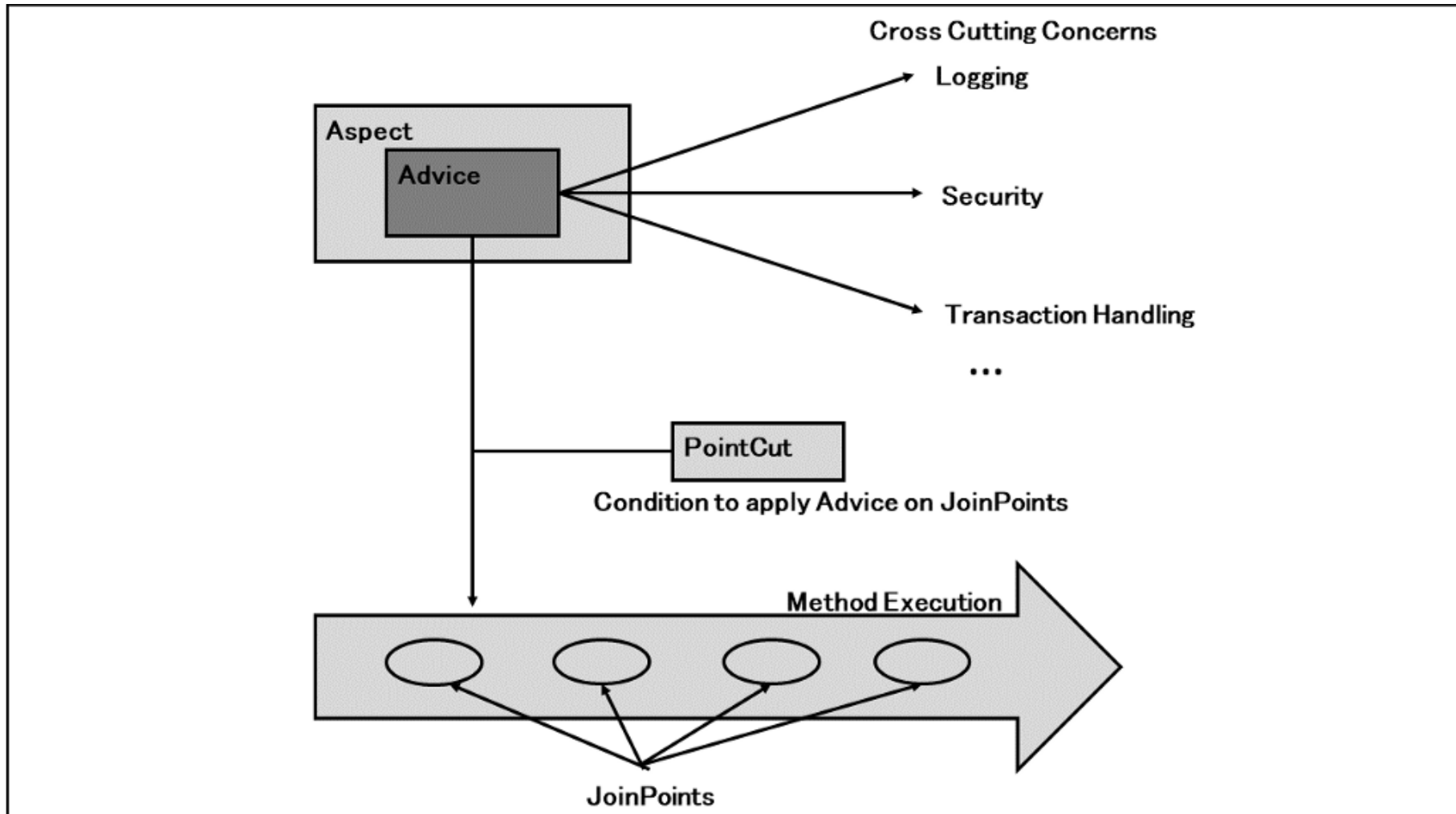
```
13 @Aspect
14 @Component
15 public class LogTimeAspect {
16
17
18     Logger log = LoggerFactory.getLogger(LogTimeAspect.class);
19
20     @Around(value = "execution(* com.global.book.service..*(..))")
21     public Object logTime(ProceedingJoinPoint joinPoint) throws Throwable {
22
23         long startTime = System.currentTimeMillis();
24         StringBuilder sb = new StringBuilder("KPI:");
25         sb.append("[").append(joinPoint.getKind()).append("]\tfor: ").append(joinPoint.getSignature())
26             .append("\twithArgs: ").append("(").append(StringUtils.join(joinPoint.getArgs(), ","))
27             .append(")");
28         sb.append("\tttook: ");
29         Object returnValue = joinPoint.proceed();
30         log.info(sb.append(System.currentTimeMillis() - startTime).append(" ms.").toString());
31
32         return returnValue;
33     }
34 }
```

- Weaving
 - Combines advices with point cuts
 - The incorporation of advice code at the specified join points by an aspect weaver.

Aspect Weaving



AOP Terminologies



**THANK
YOU!**

